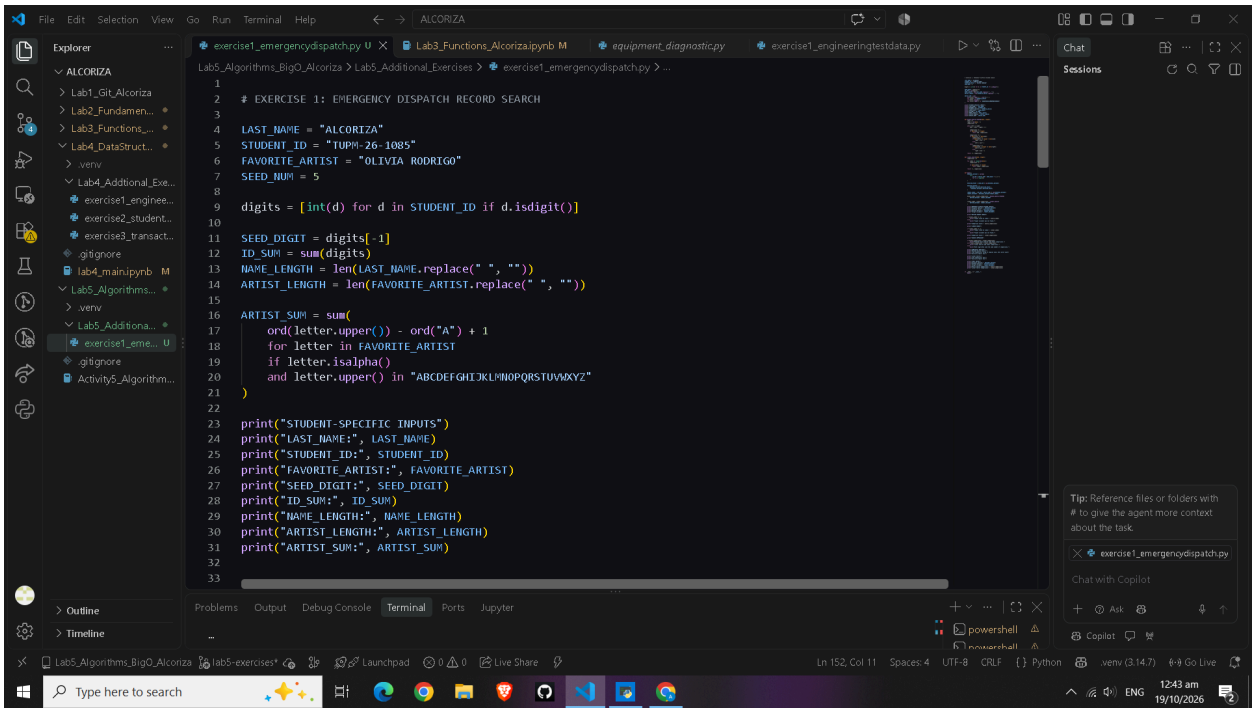
Avineth Alcoriza
BSECE -1C
COMPROG 1 - LAB 5 ADDITIONAL EXERCISES

EXERCISE 1 CODE



```python
# EXERCISE 1: EMERGENCY DISPATCH RECORD SEARCH

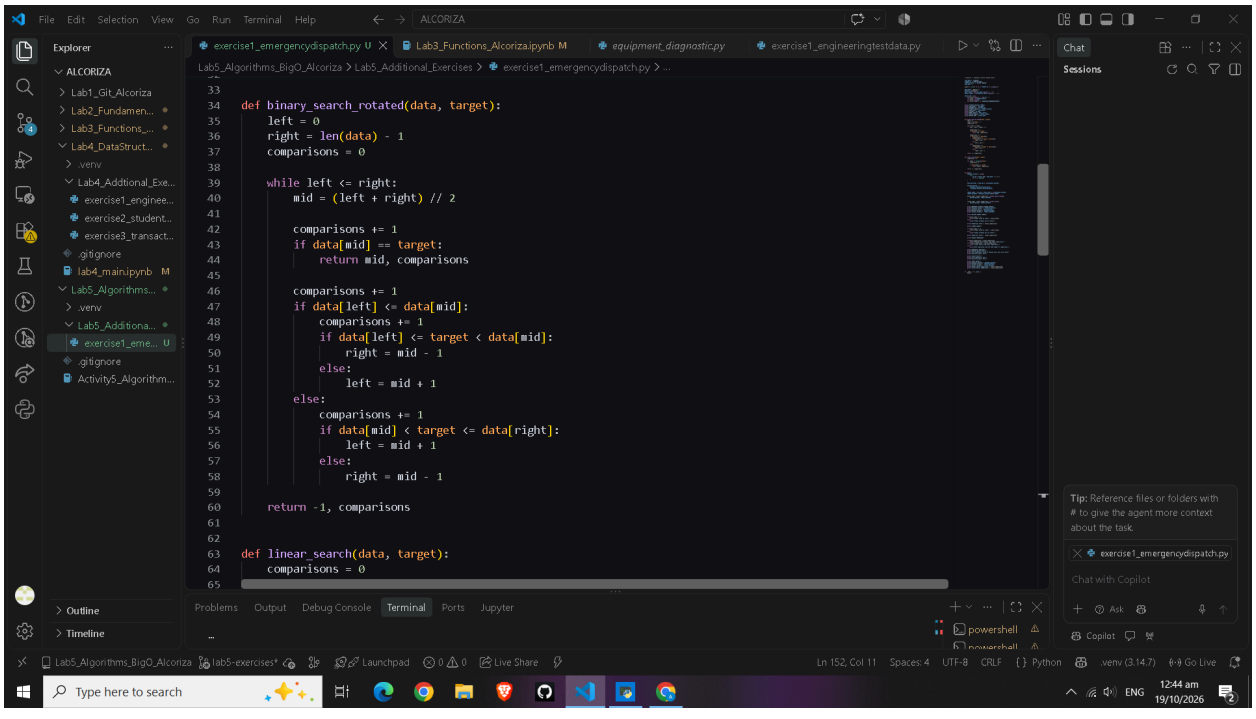
LAST_NAME = "ALCORIZA"
STUDENT_ID = "TUPM-26-1085"
FAVORITE_ARTIST = "OLIVIA RODRIGO"
SEED_NUM = 5

digits = [int(d) for d in STUDENT_ID if d.isdigit()]

SEED_DIGIT = digits[-1]
ID_SUM = sum(digits)
NAME_LENGTH = len(LAST_NAME.replace(" ", ""))
ARTIST_LENGTH = len(FAVORITE_ARTIST.replace(" ", ""))

ARTIST_SUM = sum(
    ord(letter.upper()) - ord("A") + 1
    for letter in FAVORITE_ARTIST
    if letter.isalpha()
    and letter.upper() in "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
)

print("STUDENT-SPECIFIC INPUTS")
print("LAST_NAME:", LAST_NAME)
print("STUDENT_ID:", STUDENT_ID)
print("FAVORITE_ARTIST:", FAVORITE_ARTIST)
print("SEED_DIGIT:", SEED_DIGIT)
print("ID_SUM:", ID_SUM)
print("NAME_LENGTH:", NAME_LENGTH)
print("ARTIST_LENGTH:", ARTIST_LENGTH)
print("ARTIST_SUM:", ARTIST_SUM)
```



```python
def binary_search_rotated(data, target):
    left = 0
    right = len(data) - 1
    comparisons = 0

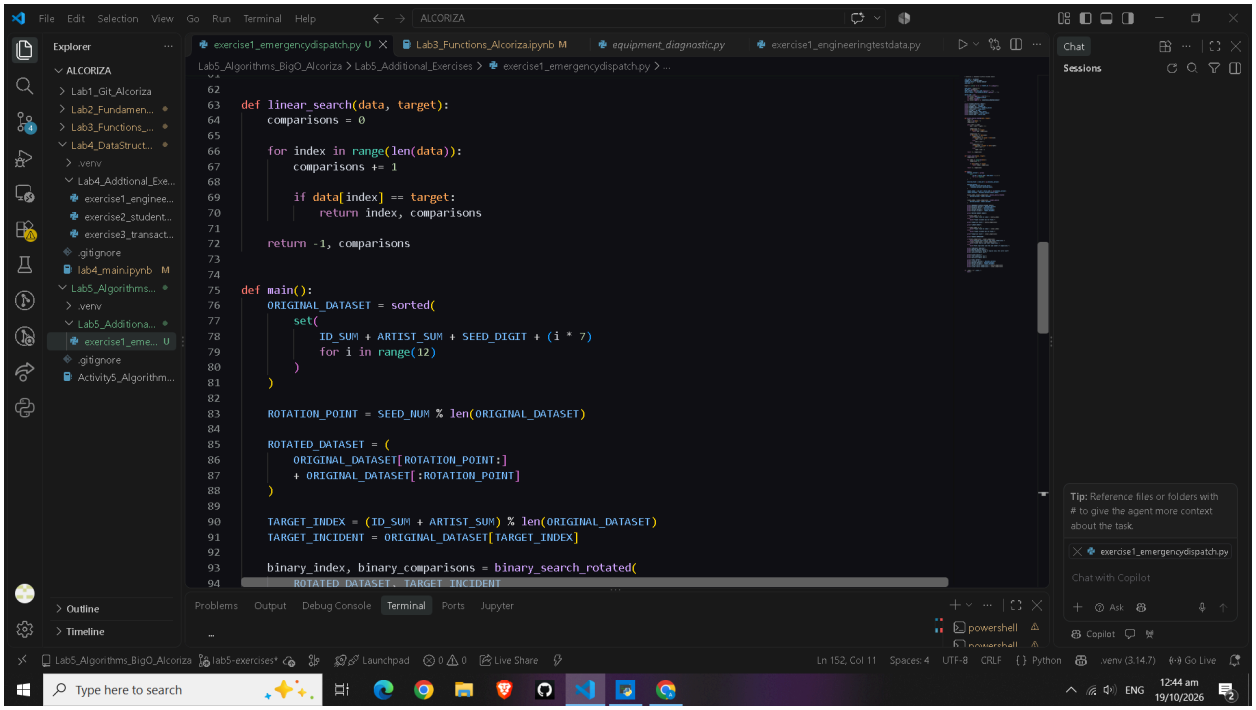
    while left <= right:
        mid = (left + right) // 2

        comparisons += 1
        if data[mid] == target:
            return mid, comparisons

        comparisons += 1
        if data[left] <= data[mid]:
            comparisons += 1
            if data[left] <= target < data[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            comparisons += 1
            if data[mid] < target <= data[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1, comparisons


def linear_search(data, target):
    comparisons = 0
```



```python
def linear_search(data, target):
    comparisons = 0

    for index in range(len(data)):
        comparisons += 1

        if data[index] == target:
            return index, comparisons

    return -1, comparisons

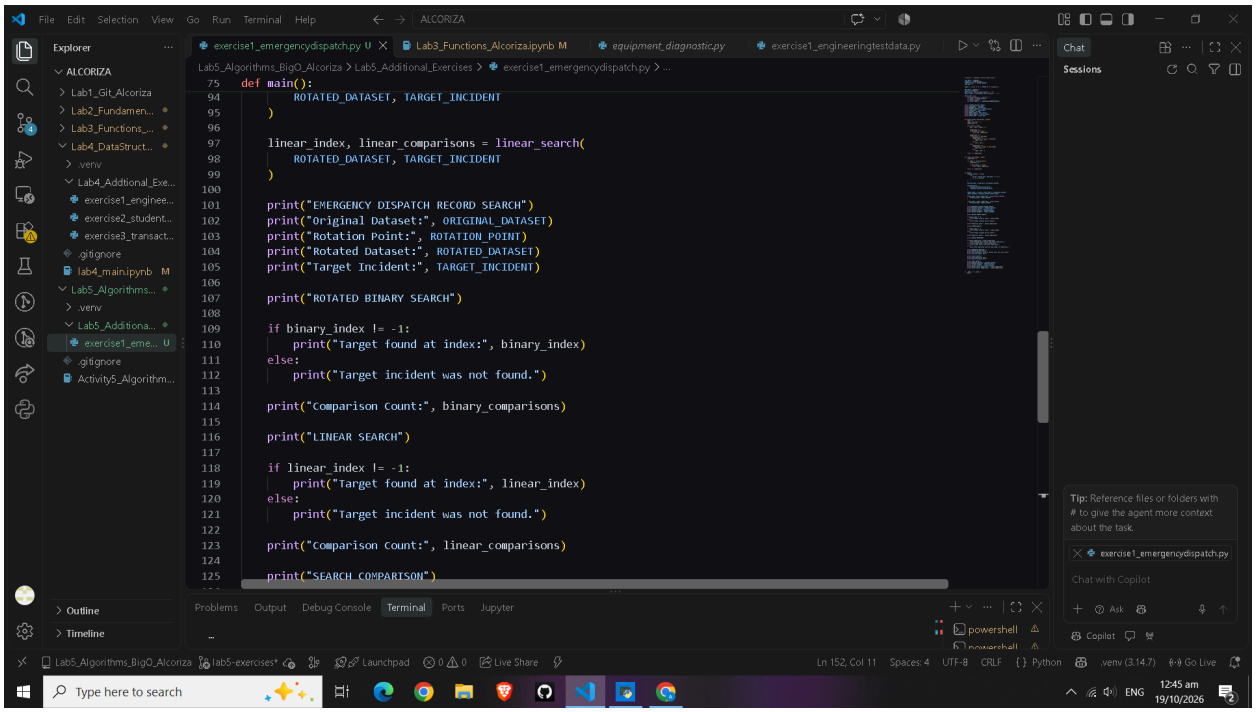

def main():
    ORIGINAL_DATASET = sorted(
        set(
            ID_SUM + ARTIST_SUM + SEED_DIGIT + (i * 7)
            for i in range(12)
        )
    )

    ROTATION_POINT = SEED_NUM % len(ORIGINAL_DATASET)

    ROTATED_DATASET = (
        ORIGINAL_DATASET[ROTATION_POINT:]
        + ORIGINAL_DATASET[:ROTATION_POINT]
    )

    TARGET_INDEX = (ID_SUM + ARTIST_SUM) % len(ORIGINAL_DATASET)
    TARGET_INCIDENT = ORIGINAL_DATASET[TARGET_INDEX]

    binary_index, binary_comparisons = binary_search_rotated(
        ROTATED_DATASET, TARGET_INCIDENT
```



```python
def main():
        ROTATED_DATASET, TARGET_INCIDENT
    )

    linear_index, linear_comparisons = linear_search(
        ROTATED_DATASET, TARGET_INCIDENT
    )

    print("EMERGENCY DISPATCH RECORD SEARCH")
    print("Original Dataset:", ORIGINAL_DATASET)
    print("Rotation Point:", ROTATION_POINT)
    print("Rotated Dataset:", ROTATED_DATASET)
    print("Target Incident:", TARGET_INCIDENT)

    print("ROTATED BINARY SEARCH")

    if binary_index != -1:
        print("Target found at index:", binary_index)
    else:
        print("Target incident was not found.")

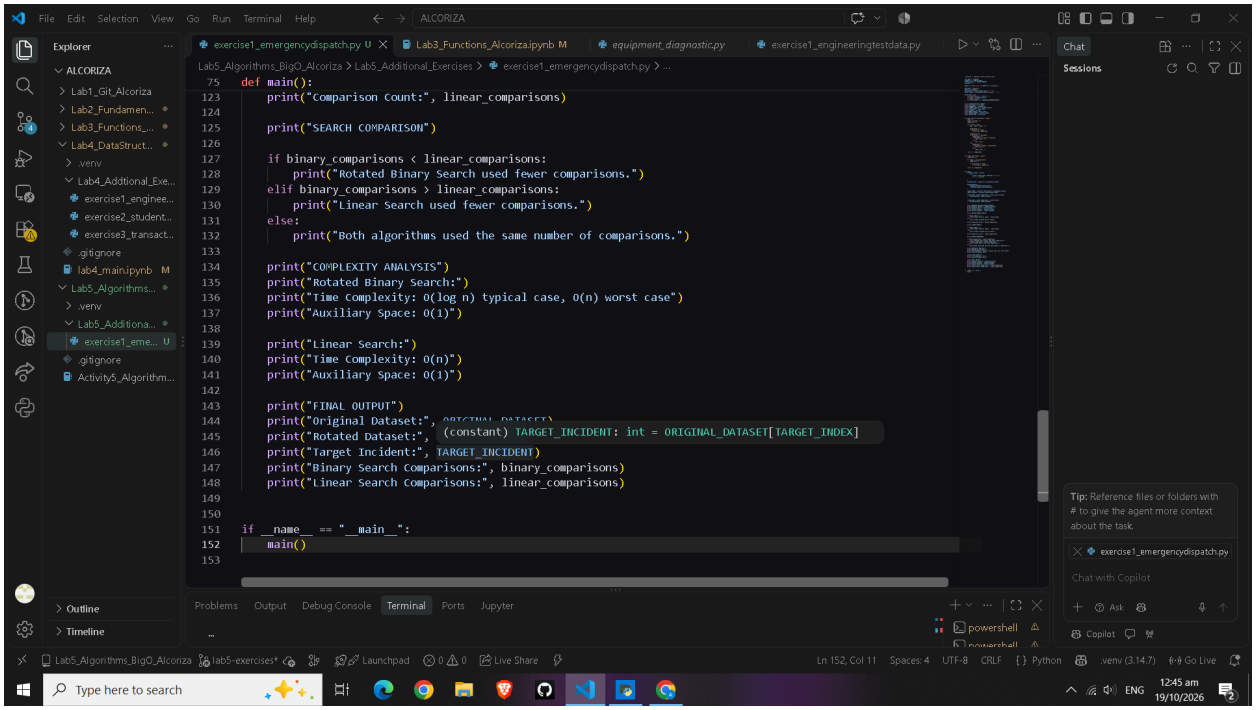
    print("Comparison Count:", binary_comparisons)

    print("LINEAR SEARCH")

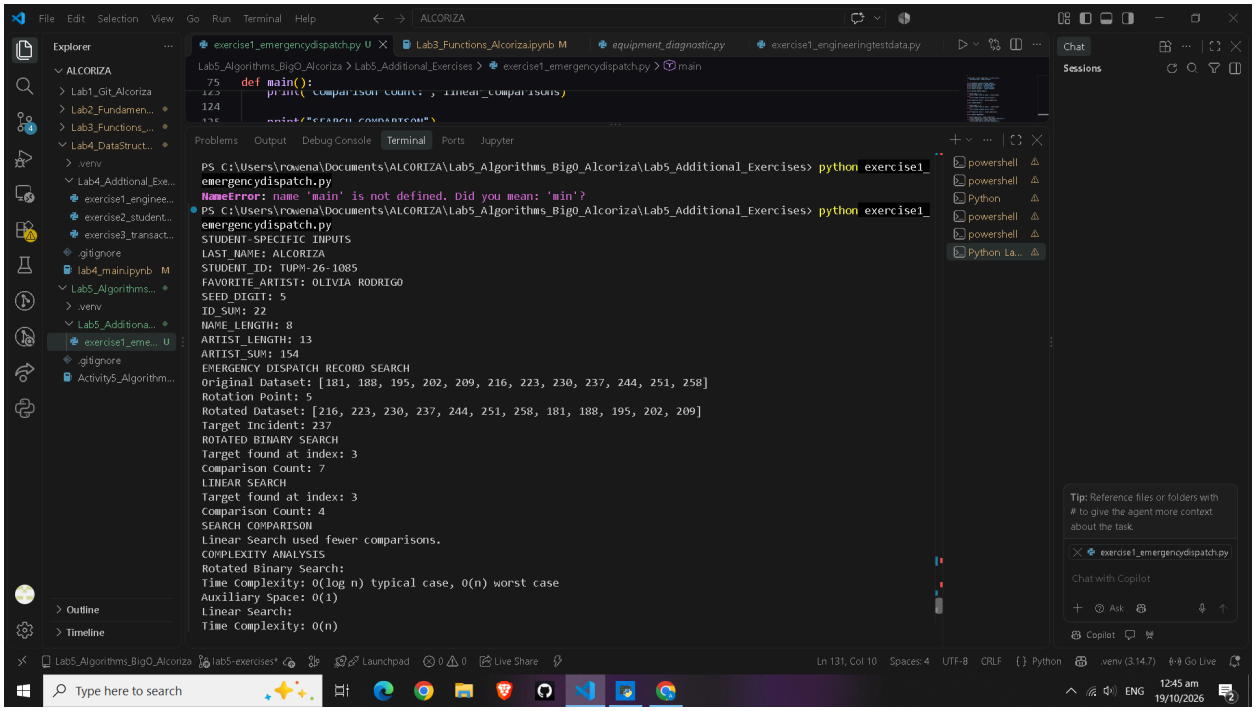
    if linear_index != -1:
        print("Target found at index:", linear_index)
    else:
        print("Target incident was not found.")

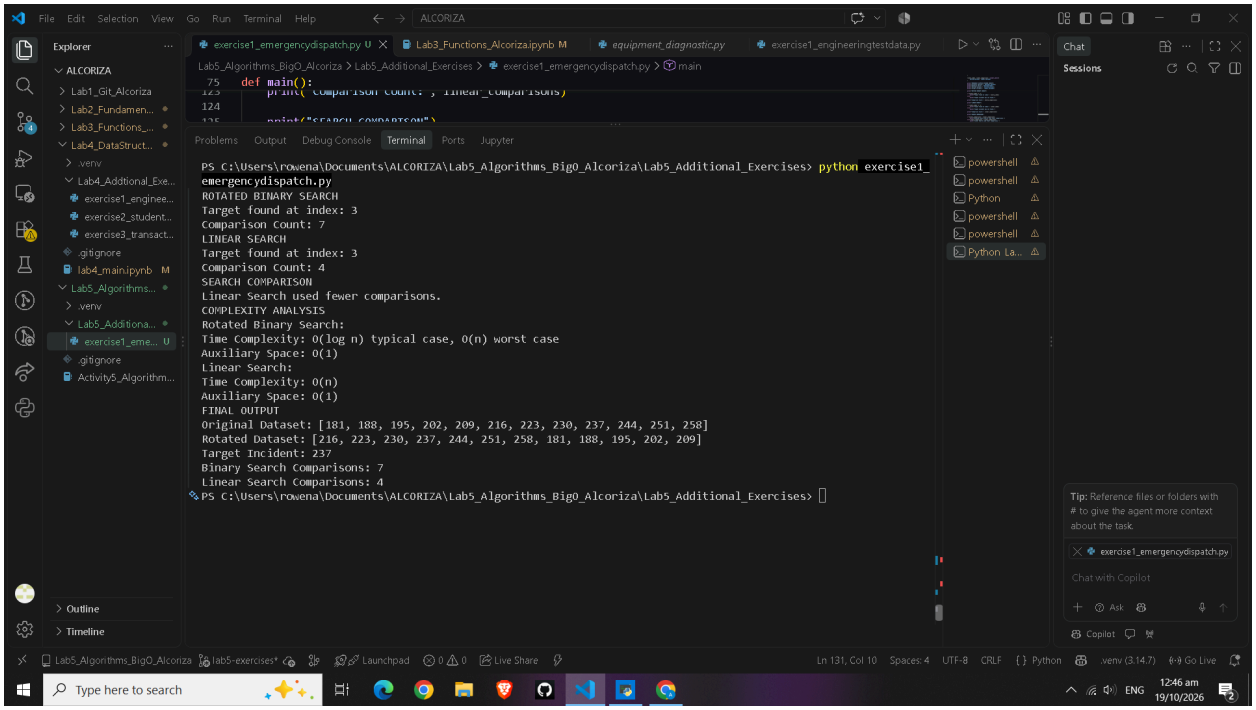
    print("Comparison Count:", linear_comparisons)

    print("SEARCH COMPARISON")
```

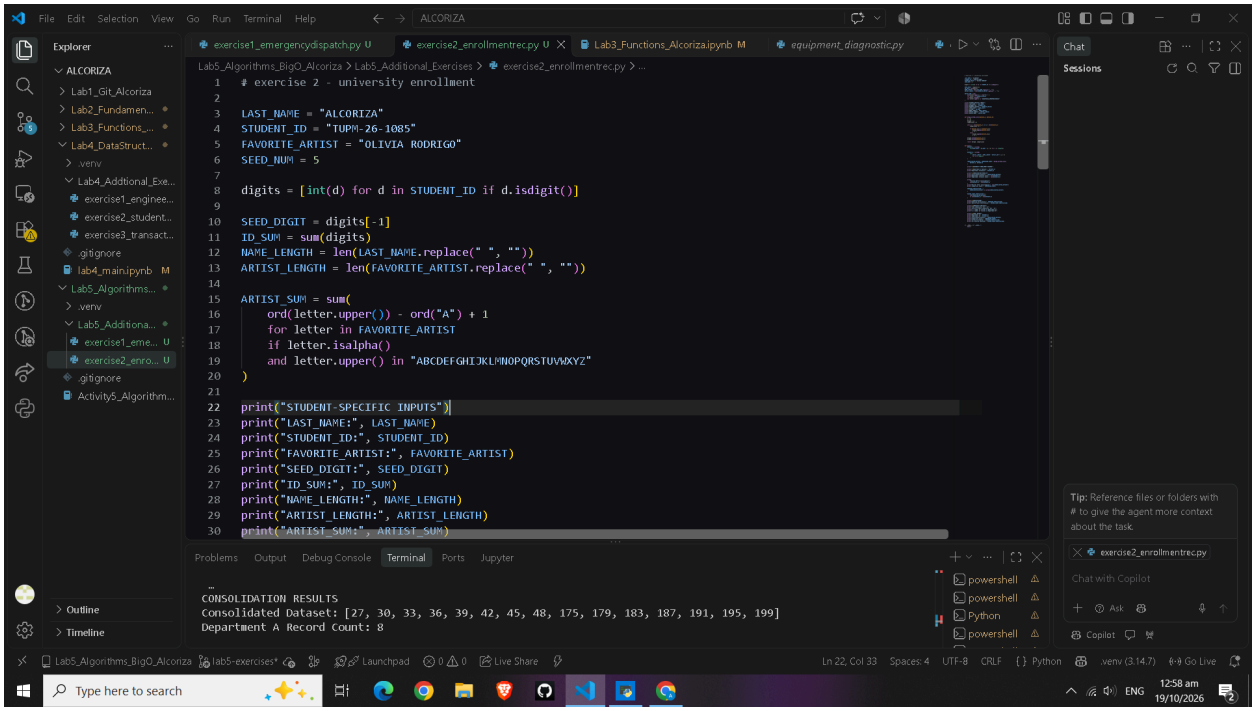

def main():
print("Comparison Count:", linear_comparisons)
print("SEARCH COMPARISON")
if binary_comparisons < linear_comparisons:
print("Rotated Binary Search used fewer comparisons.")
elif binary_comparisons > linear_comparisons:
print("Linear Search used fewer comparisons.")
else:
print("Both algorithms used the same number of comparisons.")
print("COMPLEXITY ANALYSIS")
print("Rotated Binary Search:")
print("Time Complexity: O(log n) typical case, O(n) worst case")
print("Auxiliary Space: O(1)")
print("Linear Search:")
print("Time Complexity: O(n)")
print("Auxiliary Space: O(1)")
print("FINAL OUTPUT")
print("Original Dataset:", ORIGINAL_DATASET)
print("Rotated Dataset:",
(constant) TARGET_INCIDENT: int = ORIGINAL_DATASET[TARGET_INDEX]
print("Target Incident:", TARGET_INCIDENT)
print("Binary Search Comparisons:", binary_comparisons)
print("Linear Search Comparisons:", linear_comparisons)
if __name__ == "__main__":
main()

RESULTS


PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise1_emergencydispatch.py
NameError: name 'main' is not defined. Did you mean: 'min'?
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise1_emergencydispatch.py
STUDENT-SPECIFIC INPUTS
LAST_NAME: ALCORIZA
STUDENT_ID: TUPM-26-1085
FAVORITE_ARTIST: OLIVIA RODRIGO
SEED_DIGIT: 5
ID_SUM: 22
NAME_LENGTH: 8
ARTIST_LENGTH: 13
ARTIST_SUM: 154
EMERGENCY DISPATCH RECORD SEARCH
Original Dataset: [181, 188, 195, 202, 209, 216, 223, 230, 237, 244, 251, 258]
Rotation Point: 5
Rotated Dataset: [216, 223, 230, 237, 244, 251, 258, 181, 188, 195, 202, 209]
Target Incident: 237
ROTATED BINARY SEARCH
Target found at index: 3
Comparison Count: 7
LINEAR SEARCH
Target found at index: 3
Comparison Count: 4
SEARCH COMPARISON
Linear Search used fewer comparisons.
COMPLEXITY ANALYSIS
Rotated Binary Search:
Time Complexity: O(log n) typical case, O(n) worst case
Auxiliary Space: O(1)
Linear Search:
Time Complexity: O(n)


def main():
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise1_emergencydispatch.py
ROTATED BINARY SEARCH
Target found at index: 3
Comparison Count: 7
LINEAR SEARCH
Target found at index: 3
Comparison Count: 4
SEARCH COMPARISON
Linear Search used fewer comparisons.
COMPLEXITY ANALYSIS
Rotated Binary Search:
Time Complexity: O(log n) typical case, O(n) worst case
Auxiliary Space: O(1)
Linear Search:
Time Complexity: O(n)
Auxiliary Space: O(1)
FINAL OUTPUT
Original Dataset: [181, 188, 195, 202, 209, 216, 223, 230, 237, 244, 251, 258]
Rotated Dataset: [216, 223, 230, 237, 244, 251, 258, 181, 188, 195, 202, 209]
Target Incident: 237
Binary Search Comparisons: 7
Linear Search Comparisons: 4
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises>
Type here to search

EXERCISE 2
CODE



```
# exercise 2 - university enrollment

LAST_NAME = "ALCORIZA"
STUDENT_ID = "TUPM-26-1085"
FAVORITE_ARTIST = "OLIVIA RODRIGO"
SEED_NUM = 5

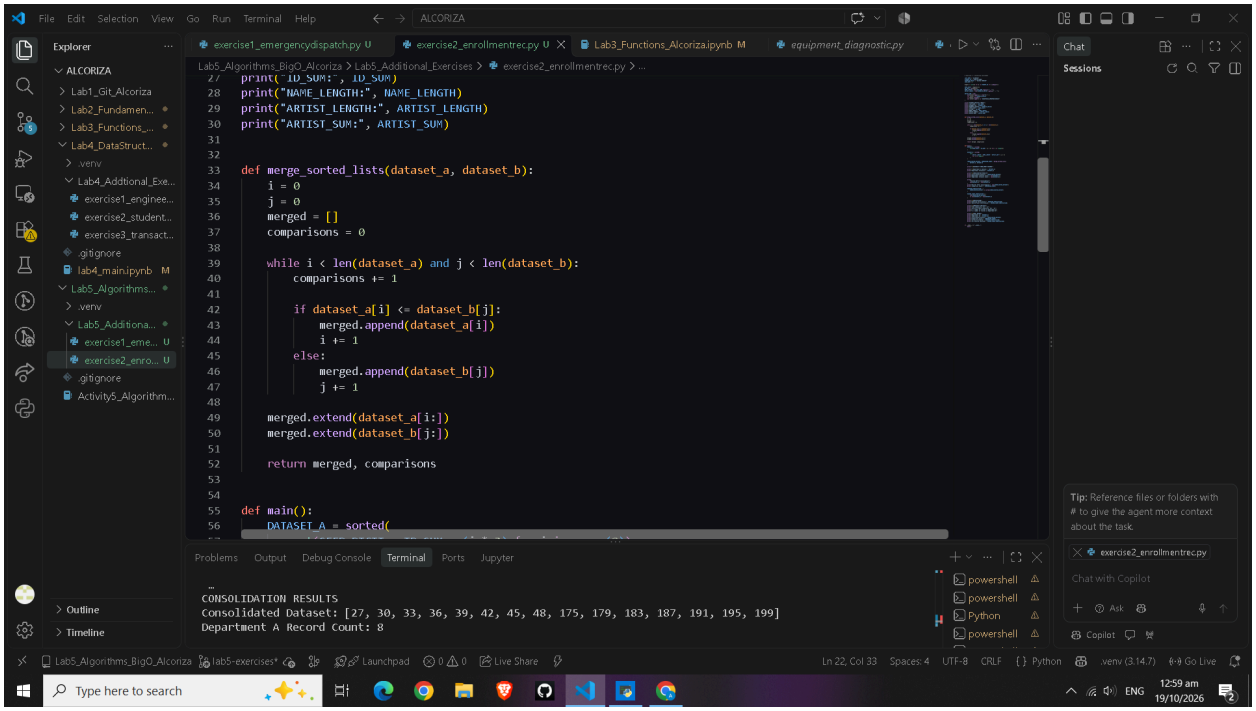
digits = [int(d) for d in STUDENT_ID if d.isdigit()]

SEED_DIGIT = digits[-1]
ID_SUM = sum(digits)
NAME_LENGTH = len(LAST_NAME.replace(" ", ""))
ARTIST_LENGTH = len(FAVORITE_ARTIST.replace(" ", ""))

ARTIST_SUM = sum(
    ord(letter.upper()) - ord("A") + 1
    for letter in FAVORITE_ARTIST
    if letter.isalpha()
    and letter.upper() in "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
)

print("STUDENT-SPECIFIC INPUTS")
print("LAST_NAME:", LAST_NAME)
print("STUDENT_ID:", STUDENT_ID)
print("FAVORITE_ARTIST:", FAVORITE_ARTIST)
print("SEED_DIGIT:", SEED_DIGIT)
print("ID_SUM:", ID_SUM)
print("NAME_LENGTH:", NAME_LENGTH)
print("ARTIST_LENGTH:", ARTIST_LENGTH)
print("ARTIST_SUM:", ARTIST_SUM)
```

```
CONSOLIDATION RESULTS
Consolidated Dataset: [27, 30, 33, 36, 39, 42, 45, 48, 175, 179, 183, 187, 191, 195, 199]
Department A Record Count: 8
```



```
print("ID_SUM:", ID_SUM)
print("NAME_LENGTH:", NAME_LENGTH)
print("ARTIST_LENGTH:", ARTIST_LENGTH)
print("ARTIST_SUM:", ARTIST_SUM)


def merge_sorted_lists(dataset_a, dataset_b):
    i = 0
    j = 0
    merged = []
    comparisons = 0

    while i < len(dataset_a) and j < len(dataset_b):
        comparisons += 1

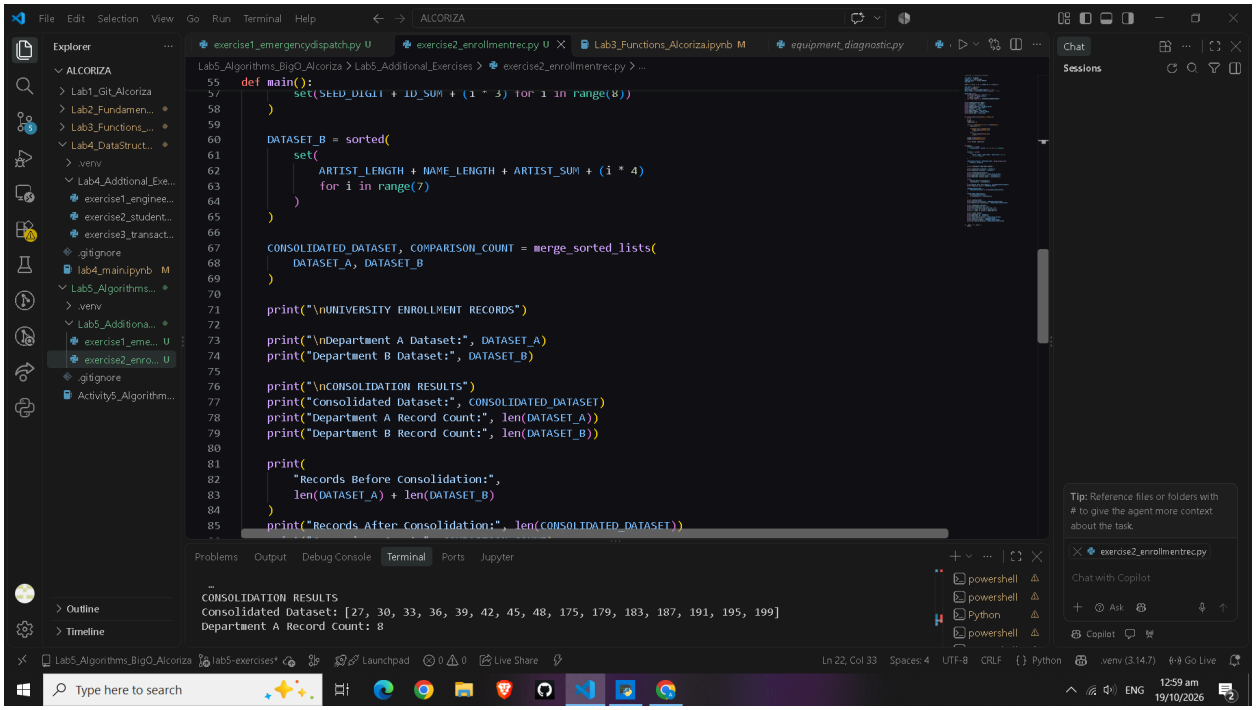
        if dataset_a[i] <= dataset_b[j]:
            merged.append(dataset_a[i])
            i += 1
        else:
            merged.append(dataset_b[j])
            j += 1

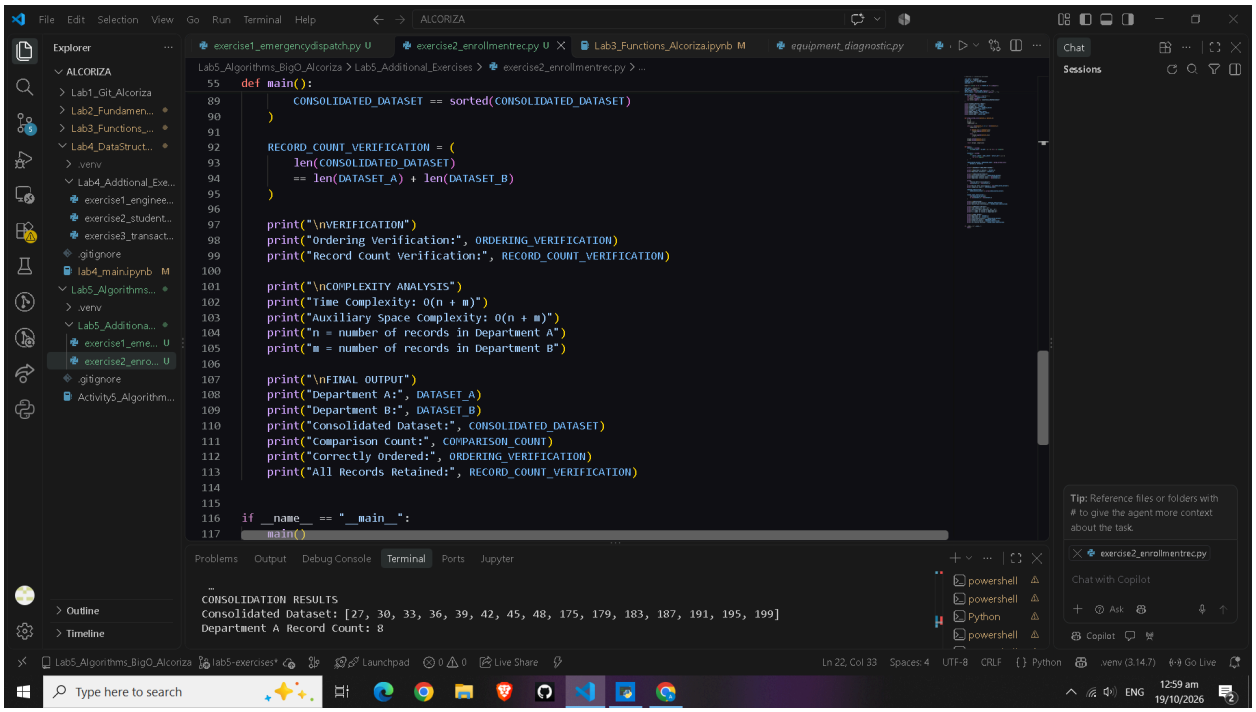
    merged.extend(dataset_a[i:])
    merged.extend(dataset_b[j:])

    return merged, comparisons

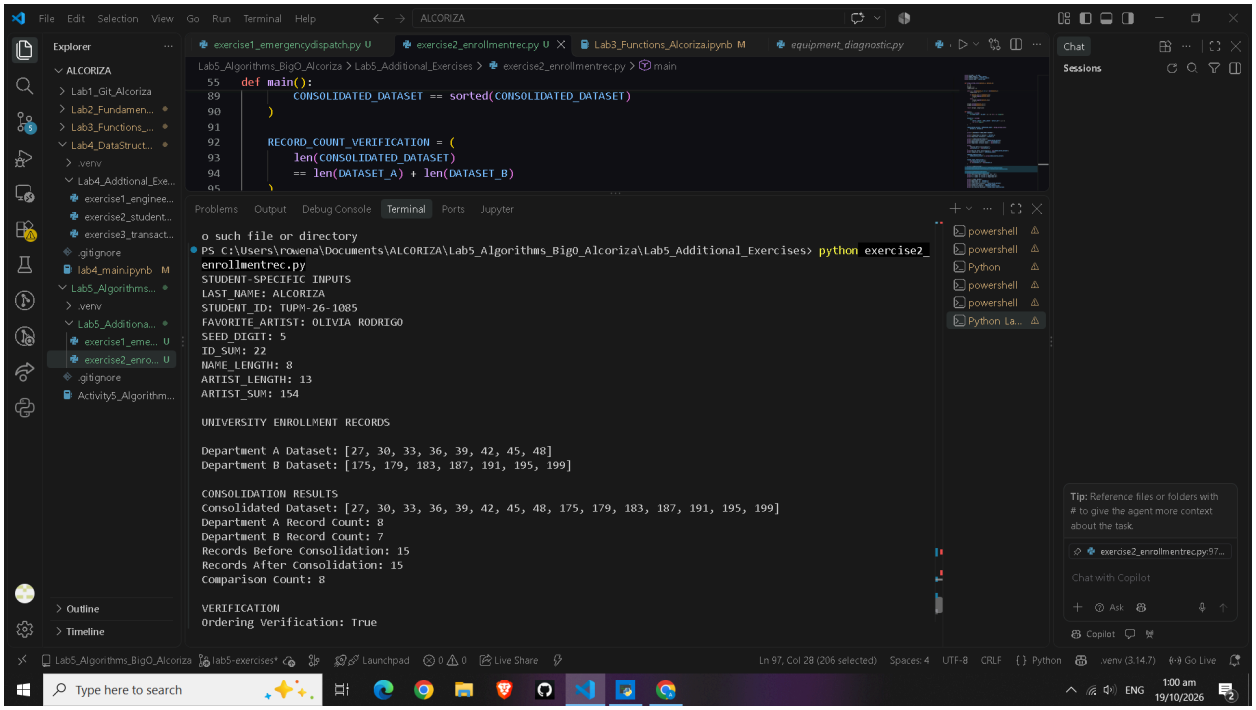

def main():
    DATASET_A = sorted(
```

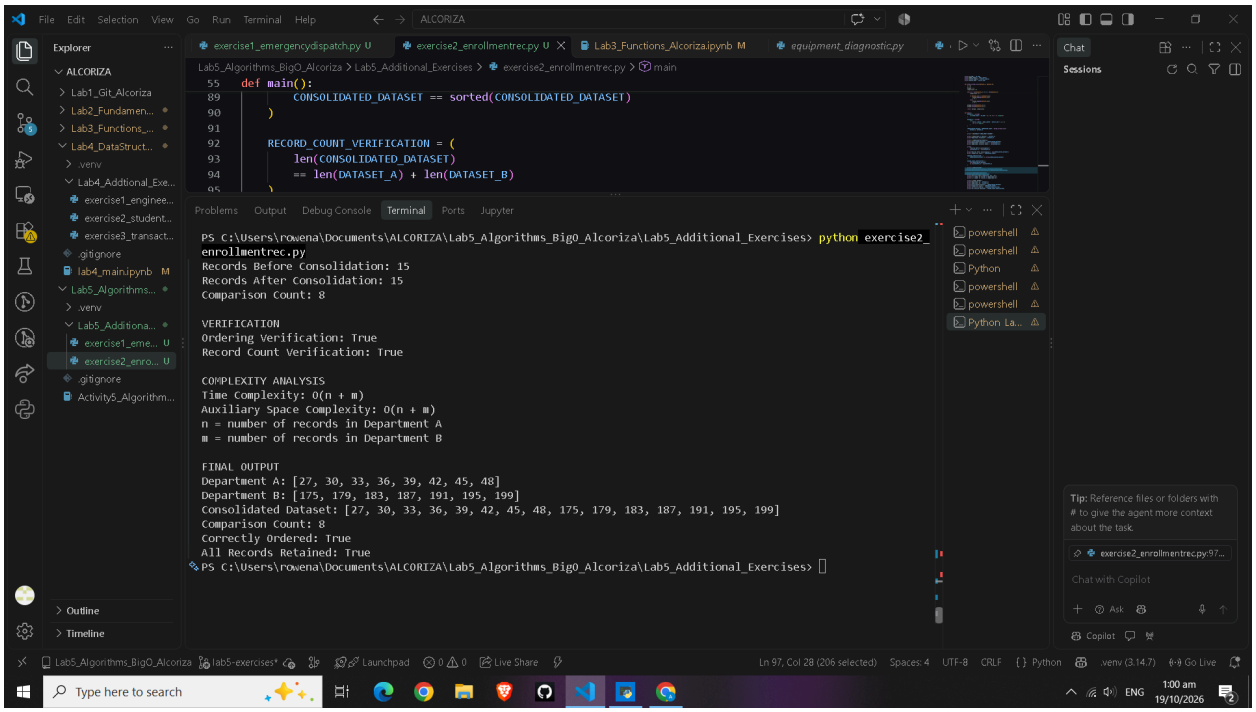
```
CONSOLIDATION RESULTS
Consolidated Dataset: [27, 30, 33, 36, 39, 42, 45, 48, 175, 179, 183, 187, 191, 195, 199]
Department A Record Count: 8
```


def main():
DATASET_B = sorted(
set(
ARTIST_LENGTH + NAME_LENGTH + ARTIST_SUM + (i * 4)
for i in range(7)
)
)
CONSOLIDATED_DATASET, COMPARISON_COUNT = merge_sorted_lists(
DATASET_A, DATASET_B
)
print("\nUNIVERSITY ENROLLMENT RECORDS")
print("\nDepartment A Dataset:", DATASET_A)
print("Department B Dataset:", DATASET_B)
print("\nCONSOLIDATION RESULTS")
print("Consolidated Dataset:", CONSOLIDATED_DATASET)
print("Department A Record Count:", len(DATASET_A))
print("Department B Record Count:", len(DATASET_B))
print(
"Records Before Consolidation:",
len(DATASET_A) + len(DATASET_B)
)
print("Records After Consolidation:", len(CONSOLIDATED_DATASET))
CONSOLIDATION RESULTS
Consolidated Dataset: [27, 30, 33, 36, 39, 42, 45, 48, 175, 179, 183, 187, 191, 195, 199]
Department A Record Count: 8

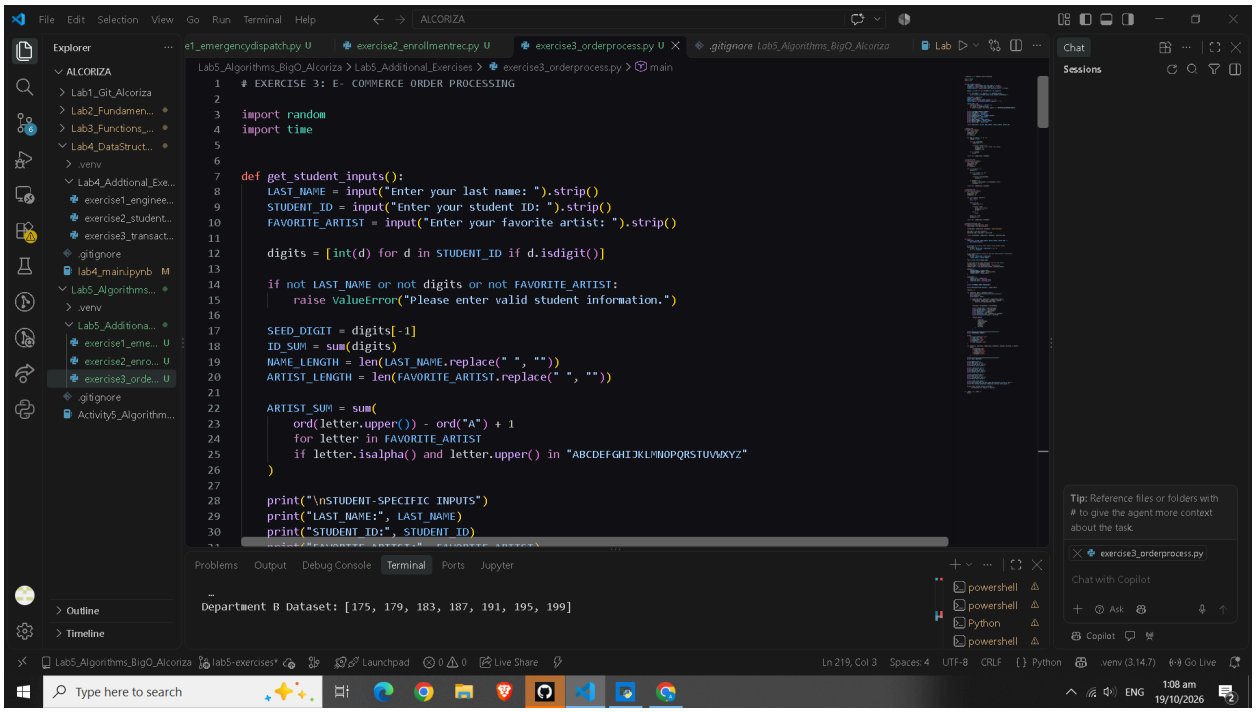

CONSOLIDATED_DATASET == sorted(CONSOLIDATED_DATASET)
RECORD_COUNT_VERIFICATION = (
len(CONSOLIDATED_DATASET)
== len(DATASET_A) + len(DATASET_B)
)
print("\nVERIFICATION")
print("Ordering Verification:", ORDERING_VERIFICATION)
print("Record Count Verification:", RECORD_COUNT_VERIFICATION)
print("\nCOMPLEXITY ANALYSIS")
print("Time Complexity: O(n + m)")
print("Auxiliary Space Complexity: O(n + m)")
print("n = number of records in Department A")
print("m = number of records in Department B")
print("\nFINAL OUTPUT")
print("Department A:", DATASET_A)
print("Department B:", DATASET_B)
print("Consolidated Dataset:", CONSOLIDATED_DATASET)
print("Comparison Count:", COMPARISON_COUNT)
print("Correctly Ordered:", ORDERING_VERIFICATION)
print("All Records Retained:", RECORD_COUNT_VERIFICATION)
if __name__ == "__main__":
main()
CONSOLIDATION RESULTS
Consolidated Dataset: [27, 30, 33, 36, 39, 42, 45, 48, 175, 179, 183, 187, 191, 195, 199]
Department A Record Count: 8

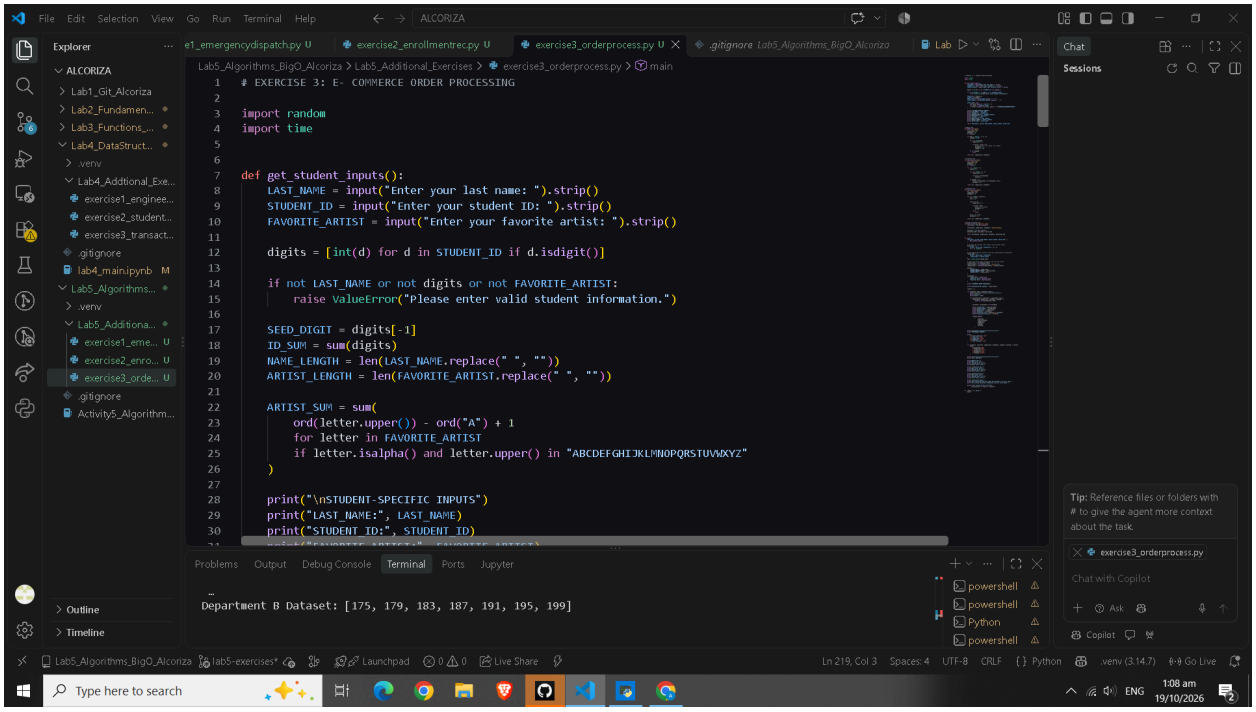
RESULTS

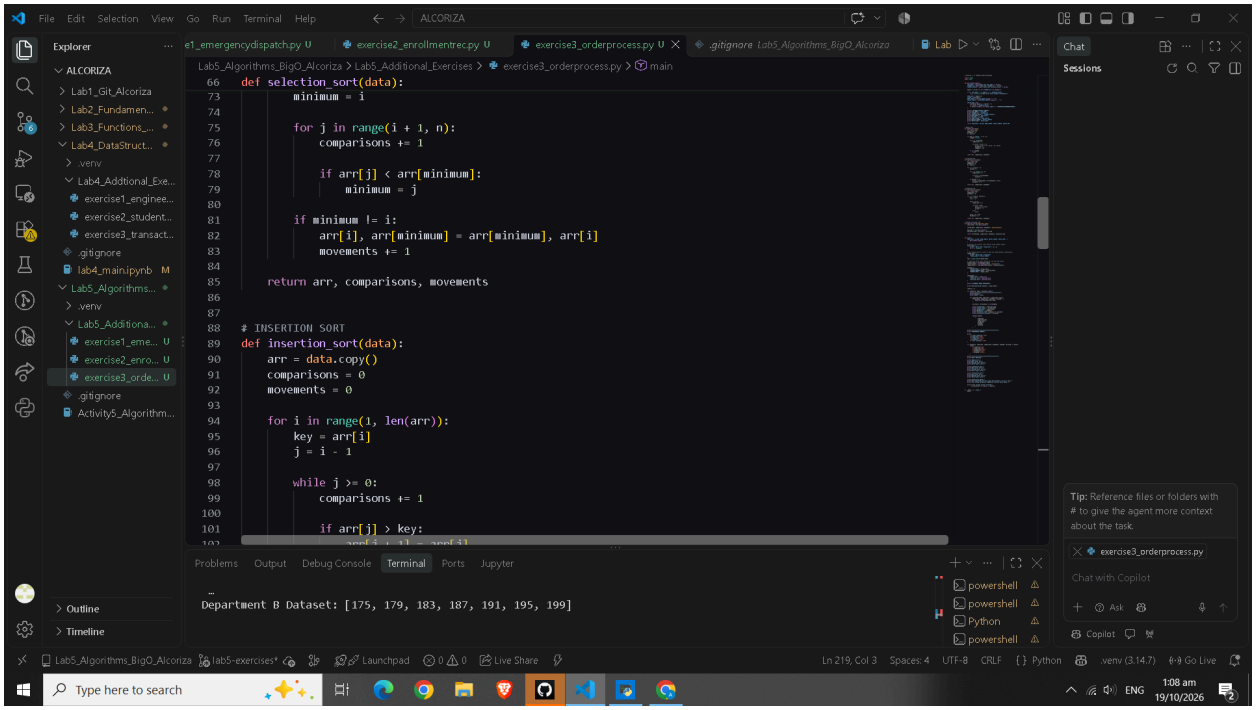

def main():
CONSOLIDATED_DATASET == sorted(CONSOLIDATED_DATASET)
RECORD_COUNT_VERIFICATION = (
len(CONSOLIDATED_DATASET)
== len(DATASET_A) + len(DATASET_B)
o such file or directory
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise2_enrollmentrec.py
STUDENT-SPECIFIC INPUTS
LAST_NAME: ALCORIZA
STUDENT_ID: TUPM-26-1085
FAVORITE_ARTIST: OLIVIA RODRIGO
SEED_DIGIT: 5
ID_SUM: 22
NAME_LENGTH: 8
ARTIST_LENGTH: 13
ARTIST_SUM: 154
UNIVERSITY ENROLLMENT RECORDS
Department A Dataset: [27, 30, 33, 36, 39, 42, 45, 48]
Department B Dataset: [175, 179, 183, 187, 191, 195, 199]
CONSOLIDATION RESULTS
Consolidated Dataset: [27, 30, 33, 36, 39, 42, 45, 48, 175, 179, 183, 187, 191, 195, 199]
Department A Record Count: 8
Department B Record Count: 7
Records Before Consolidation: 15
Records After Consolidation: 15
Comparison Count: 8
VERIFICATION
Ordering Verification: True

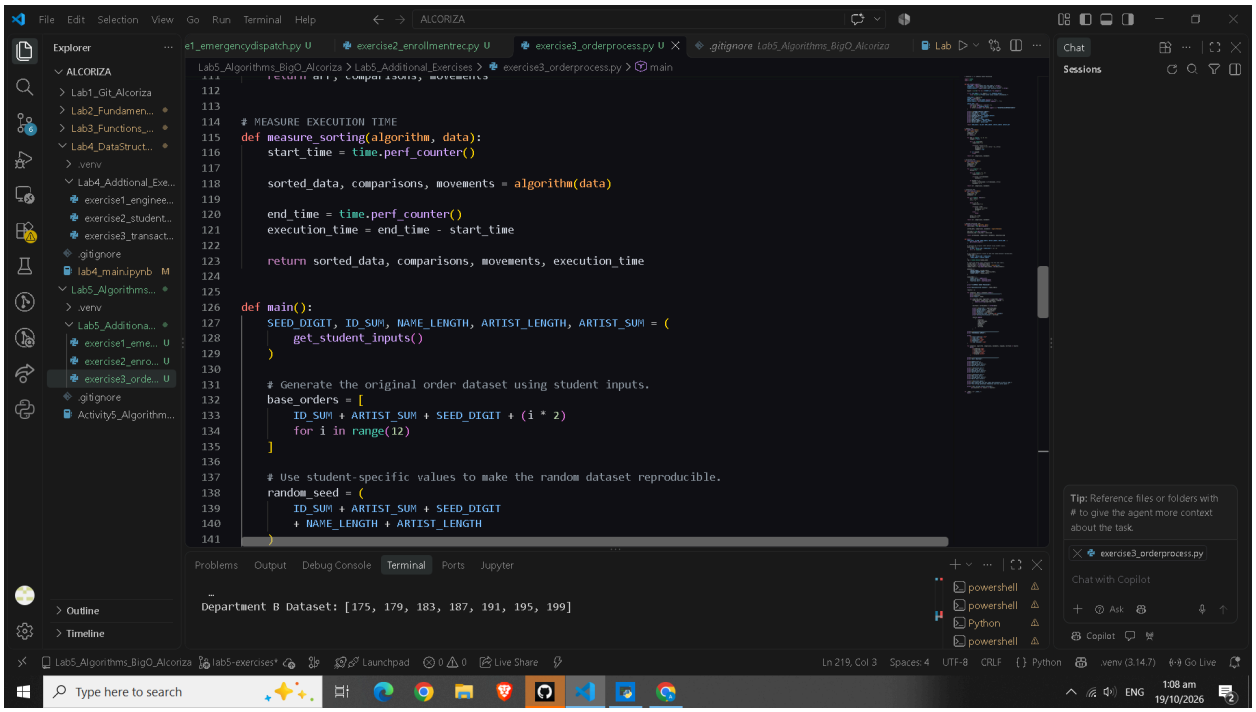

def main():
CONSOLIDATED_DATASET == sorted(CONSOLIDATED_DATASET)
RECORD_COUNT_VERIFICATION = (
len(CONSOLIDATED_DATASET)
== len(DATASET_A) + len(DATASET_B)
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise2_enrollmentrec.py
Records Before Consolidation: 15
Records After Consolidation: 15
Comparison Count: 8
VERIFICATION
Ordering Verification: True
Record Count Verification: True
COMPLEXITY ANALYSIS
Time Complexity: O(n + m)
Auxiliary Space Complexity: O(n + m)
n = number of records in Department A
m = number of records in Department B
FINAL OUTPUT
Department A: [27, 30, 33, 36, 39, 42, 45, 48]
Department B: [175, 179, 183, 187, 191, 195, 199]
Consolidated Dataset: [27, 30, 33, 36, 39, 42, 45, 48, 175, 179, 183, 187, 191, 195, 199]
Comparison Count: 8
Correctly Ordered: True
All Records Retained: True
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises>

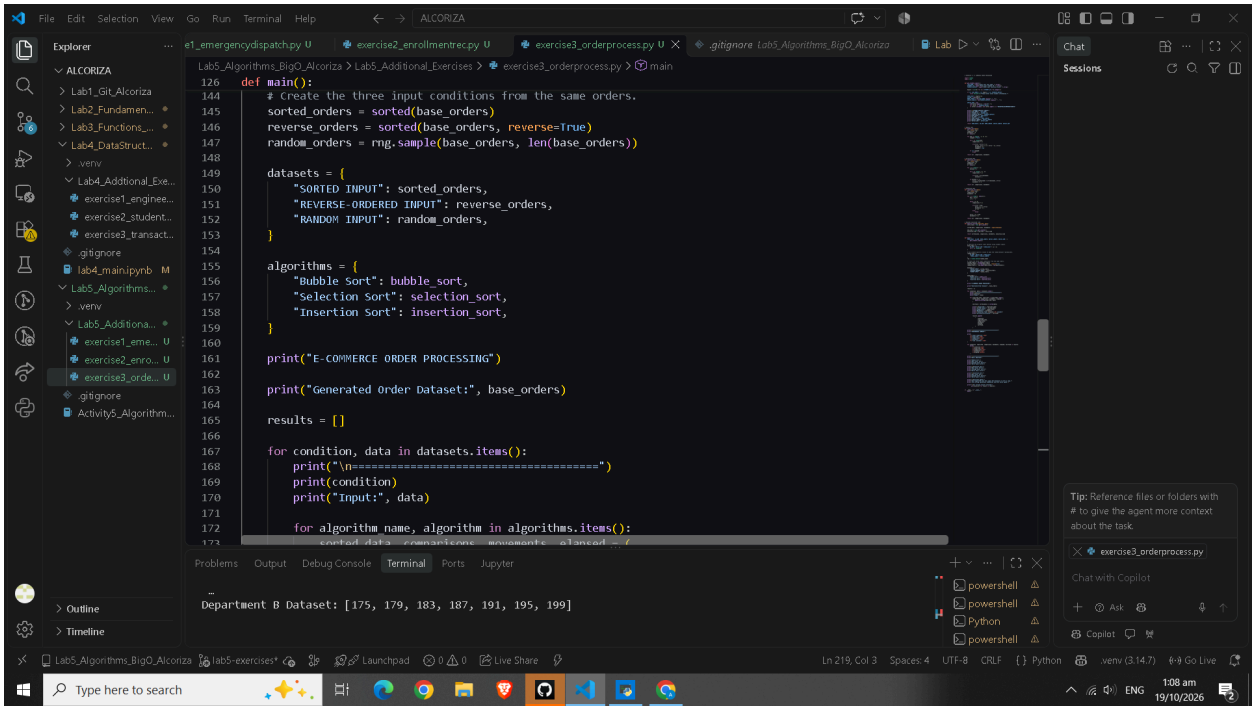
EXERCISE 3

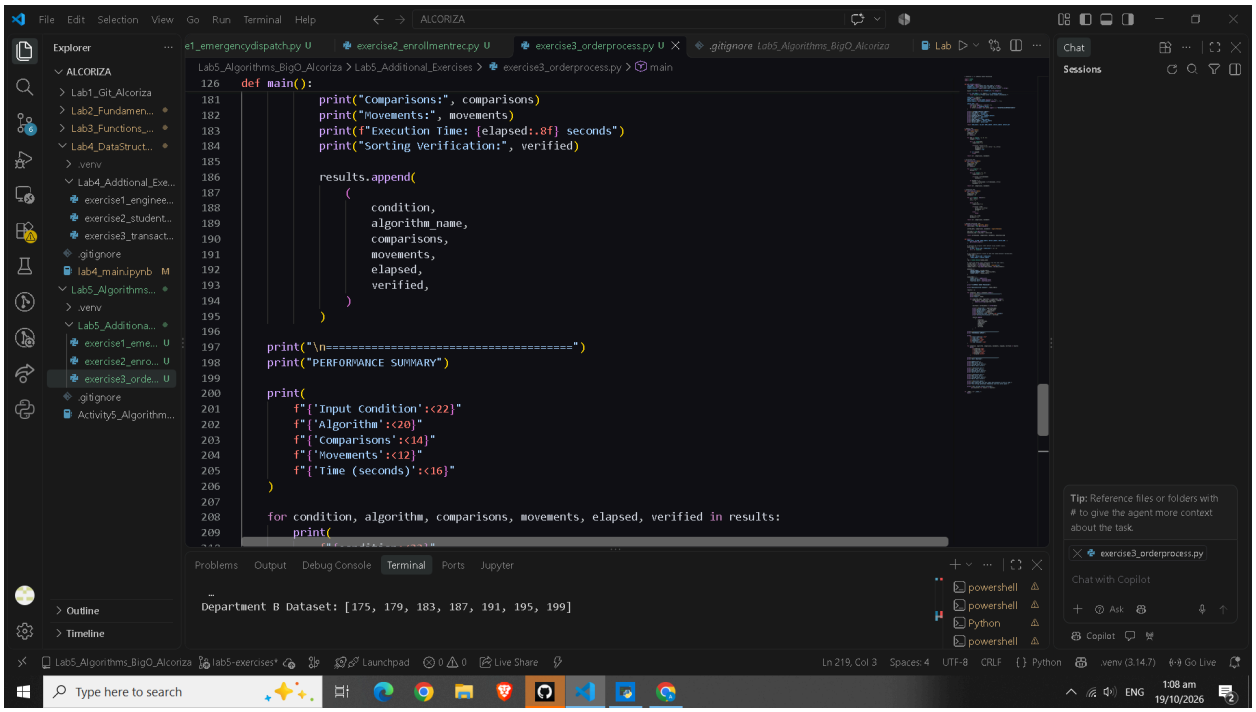

File Edit Selection View Go Run Terminal Help
ALCORIZA
Explorer
e1_emergencydispatch.py U
exercise2_enrollmentrec.py U
exercise3_orderprocess.py U
.gitignore Lab5_Algorithms_BigO_Alcoriza
Lab
Chat
Sessions
Lab5_Algorithms_BigO_Alcoriza > Lab5_Additional_Exercises > exercise3_orderprocess.py > main
EXERCISE 3: E- COMMERCE ORDER PROCESSING
import random
import time
def get_student_inputs():
LAST_NAME = input("Enter your last name: ").strip()
STUDENT_ID = input("Enter your student ID: ").strip()
FAVORITE_ARTIST = input("Enter your favorite artist: ").strip()
digits = [int(d) for d in STUDENT_ID if d.isdigit()]
if not LAST_NAME or not digits or not FAVORITE_ARTIST:
raise ValueError("Please enter valid student information.")
SEED_DIGIT = digits[-1]
ID_SUM = sum(digits)
NAME_LENGTH = len(LAST_NAME.replace(" ", ""))
ARTIST_LENGTH = len(FAVORITE_ARTIST.replace(" ", ""))
ARTIST_SUM = sum(
ord(letter.upper()) - ord("A") + 1
for letter in FAVORITE_ARTIST
if letter.isalpha() and letter.upper() in "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
)
print("\nSTUDENT-SPECIFIC INPUTS")
print("LAST_NAME:", LAST_NAME)
print("STUDENT_ID:", STUDENT_ID)
Problems Output Debug Console Terminal Ports Jupyter
Department B Dataset: [175, 179, 183, 187, 191, 195, 199]
powershell
powershell
Python
powershell
Tip: Reference files or folders with # to give the agent more context about the task.
exercise3_orderprocess.py
Chat with Copilot
Ask
Copilot
Outline
Timeline
Ln 219, Col 3 Spaces: 4 UTF-8 CRLF Python .venv (3.14.7) Go Live
Type here to search
ENG
1:08 am
19/10/2026

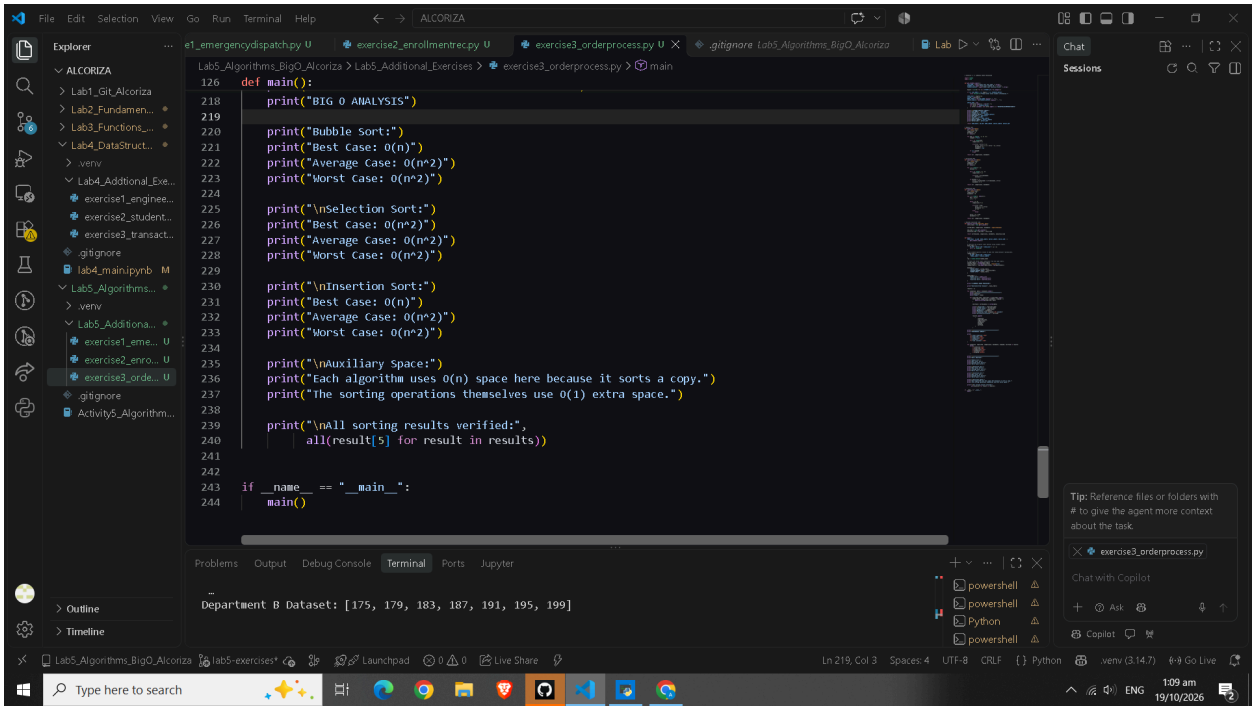

File Edit Selection View Go Run Terminal Help
ALCORIZA
Explorer
e1_emergencydispatch.py U
exercise2_enrollmentrec.py U
exercise3_orderprocess.py U
.gitignore Lab5_Algorithms_BigO_Alcoriza
Lab
Chat
Sessions
Lab5_Algorithms_BigO_Alcoriza > Lab5_Additional_Exercises > exercise3_orderprocess.py > main
EXERCISE 3: E- COMMERCE ORDER PROCESSING
import random
import time
def get_student_inputs():
LAST_NAME = input("Enter your last name: ").strip()
STUDENT_ID = input("Enter your student ID: ").strip()
FAVORITE_ARTIST = input("Enter your favorite artist: ").strip()
digits = [int(d) for d in STUDENT_ID if d.isdigit()]
if not LAST_NAME or not digits or not FAVORITE_ARTIST:
raise ValueError("Please enter valid student information.")
SEED_DIGIT = digits[-1]
ID_SUM = sum(digits)
NAME_LENGTH = len(LAST_NAME.replace(" ", ""))
ARTIST_LENGTH = len(FAVORITE_ARTIST.replace(" ", ""))
ARTIST_SUM = sum(
ord(letter.upper()) - ord("A") + 1
for letter in FAVORITE_ARTIST
if letter.isalpha() and letter.upper() in "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
)
print("\nSTUDENT-SPECIFIC INPUTS")
print("LAST_NAME:", LAST_NAME)
print("STUDENT_ID:", STUDENT_ID)
Problems Output Debug Console Terminal Ports Jupyter
Department B Dataset: [175, 179, 183, 187, 191, 195, 199]
powershell
powershell
Python
powershell
Tip: Reference files or folders with # to give the agent more context about the task.
exercise3_orderprocess.py
Chat with Copilot
Ask
Copilot
Outline
Timeline
Ln 219, Col 3 Spaces: 4 UTF-8 CRLF Python .venv (3.14.7) Go Live
Type here to search
ENG
1:08 am
19/10/2026


def selection_sort(data):
minimum = i
for j in range(i + 1, n):
comparisons += 1
if arr[j] < arr[minimum]:
minimum = j
if minimum != i:
arr[i], arr[minimum] = arr[minimum], arr[i]
movements += 1
return arr, comparisons, movements
INSERTION SORT
def insertion_sort(data):
arr = data.copy()
comparisons = 0
movements = 0
for i in range(1, len(arr)):
key = arr[i]
j = i - 1
while j >= 0:
comparisons += 1
if arr[j] > key:
Department B Dataset: [175, 179, 183, 187, 191, 195, 199]

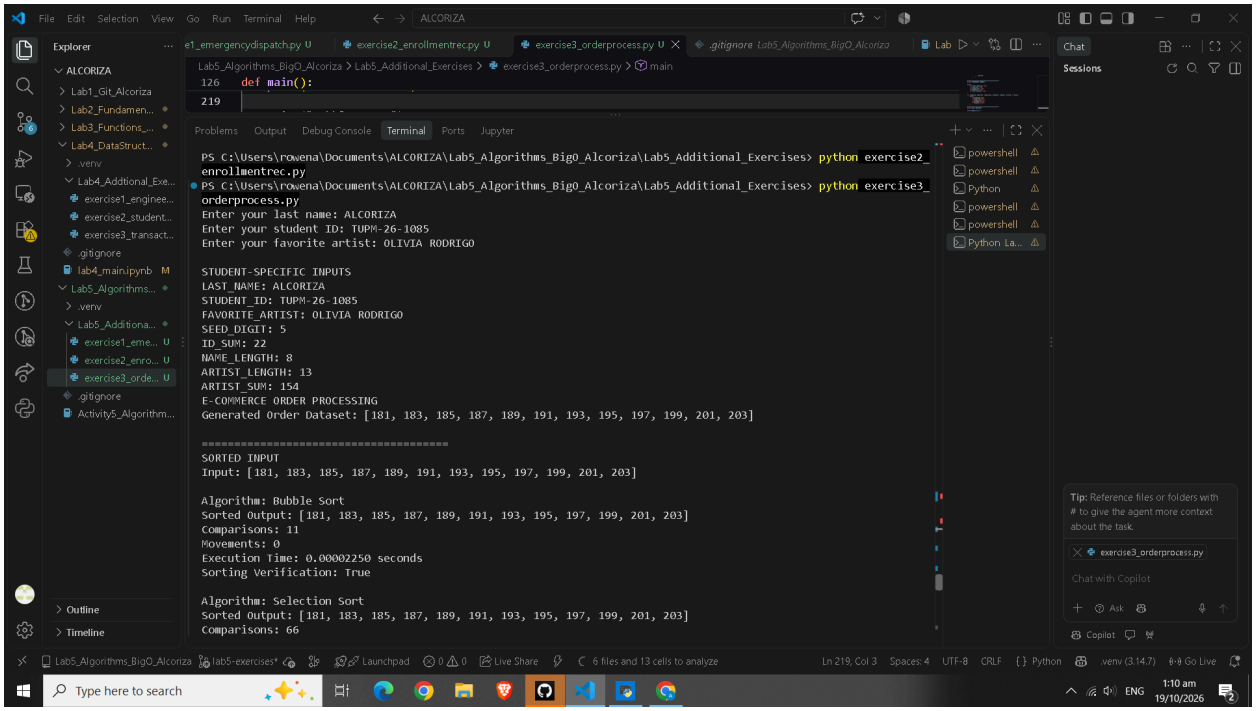

MEASURE EXECUTION TIME
def measure_sorting(algorithm, data):
start_time = time.perf_counter()
sorted_data, comparisons, movements = algorithm(data)
end_time = time.perf_counter()
execution_time = end_time - start_time
return sorted_data, comparisons, movements, execution_time
def main():
SEED_DIGIT, ID_SUM, NAME_LENGTH, ARTIST_LENGTH, ARTIST_SUM = (
get_student_inputs()
)
Generate the original order dataset using student inputs.
base_orders = [
ID_SUM + ARTIST_SUM + SEED_DIGIT + (i * 2)
for i in range(12)
]
Use student-specific values to make the random dataset reproducible.
random_seed = (
ID_SUM + ARTIST_SUM + SEED_DIGIT
+ NAME_LENGTH + ARTIST_LENGTH
)
Department B Dataset: [175, 179, 183, 187, 191, 195, 199]

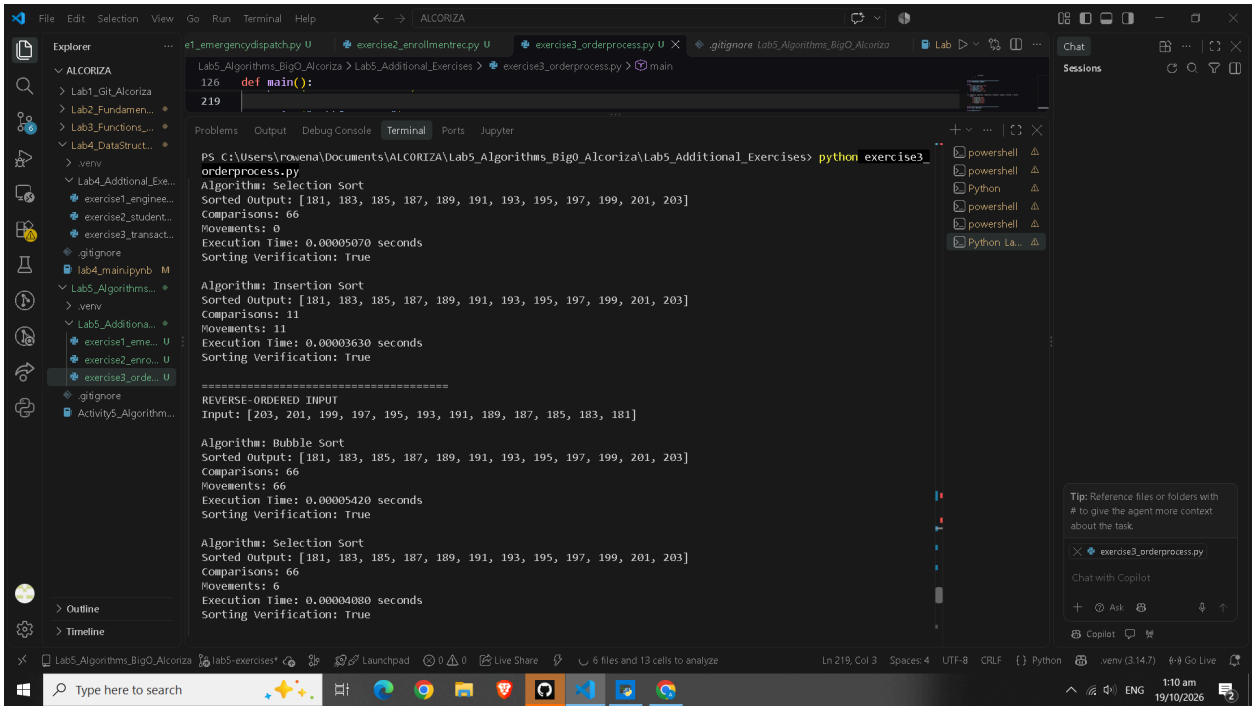

def main():
Create the three input conditions from the same orders.
sorted_orders = sorted(base_orders)
reverse_orders = sorted(base_orders, reverse=True)
random_orders = rng.sample(base_orders, len(base_orders))
datasets = {
"SORTED INPUT": sorted_orders,
"REVERSE-ORDERED INPUT": reverse_orders,
"RANDOM INPUT": random_orders,
}
algorithms = {
"Bubble Sort": bubble_sort,
"Selection Sort": selection_sort,
"Insertion Sort": insertion_sort,
}
print("E-COMMERCE ORDER PROCESSING")
print("Generated Order Dataset:", base_orders)
results = []
for condition, data in datasets.items():
print("\n====================================")
print(condition)
print("Input:", data)
for algorithm_name, algorithm in algorithms.items():
Department B Dataset: [175, 179, 183, 187, 191, 195, 199]

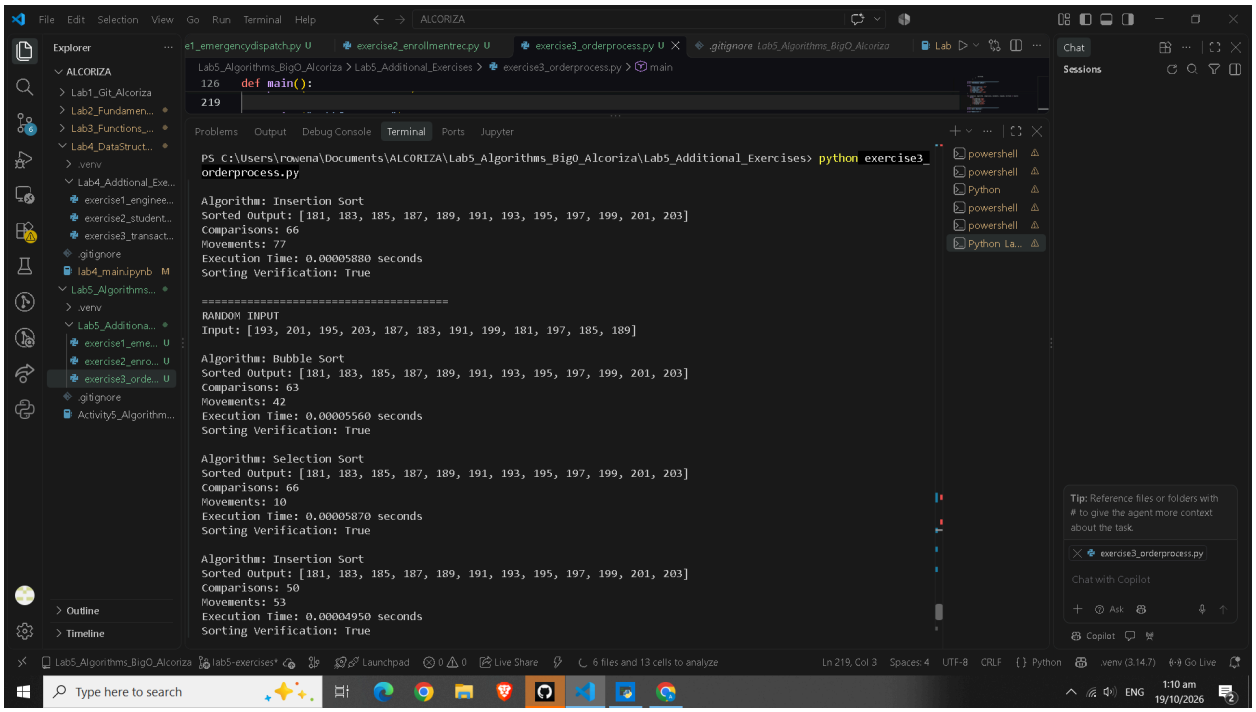

def main():
print("Comparisons:", comparisons)
print("Movements:", movements)
print(f"Execution Time: {elapsed:.8f} seconds")
print("Sorting Verification:", verified)
results.append(
(
condition,
algorithm_name,
comparisons,
movements,
elapsed,
verified,
)
)
print("\n====================================")
print("PERFORMANCE SUMMARY")
print(
f"{'Input Condition':<22}"
f"{'Algorithm':<20}"
f"{'Comparisons':<14}"
f"{'Movements':<12}"
f"{'Time (seconds)':<16}"
)
for condition, algorithm, comparisons, movements, elapsed, verified in results:
print(
Department B Dataset: [175, 179, 183, 187, 191, 195, 199]

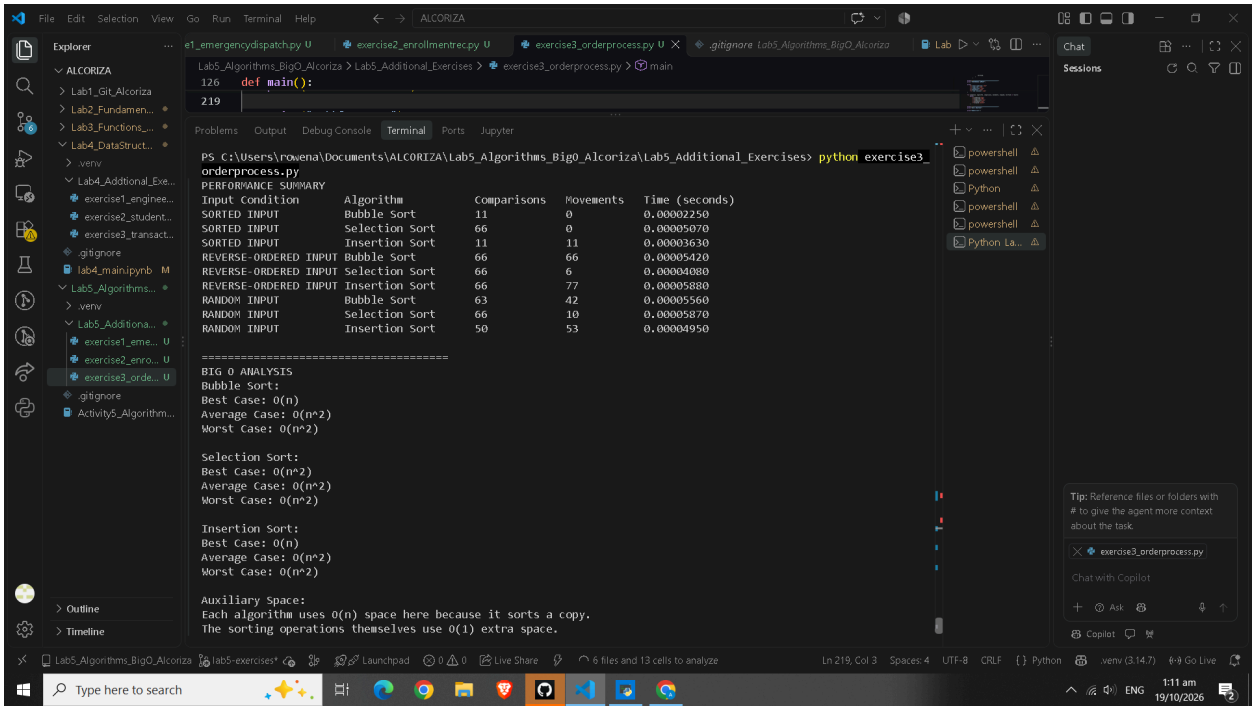

def main():
print("BIG O ANALYSIS")
print("Bubble Sort:")
print("Best Case: O(n)")
print("Average Case: O(n^2)")
print("Worst Case: O(n^2)")
print("\nSelection Sort:")
print("Best Case: O(n^2)")
print("Average Case: O(n^2)")
print("Worst Case: O(n^2)")
print("\nInsertion Sort:")
print("Best Case: O(n)")
print("Average Case: O(n^2)")
print("Worst Case: O(n^2)")
print("\nAuxiliary Space:")
print("Each algorithm uses O(n) space here because it sorts a copy.")
print("The sorting operations themselves use O(1) extra space.")
print("\nAll sorting results verified:",
all(result[5] for result in results))
if __name__ == "__main__":
main()
Department B Dataset: [175, 179, 183, 187, 191, 195, 199]

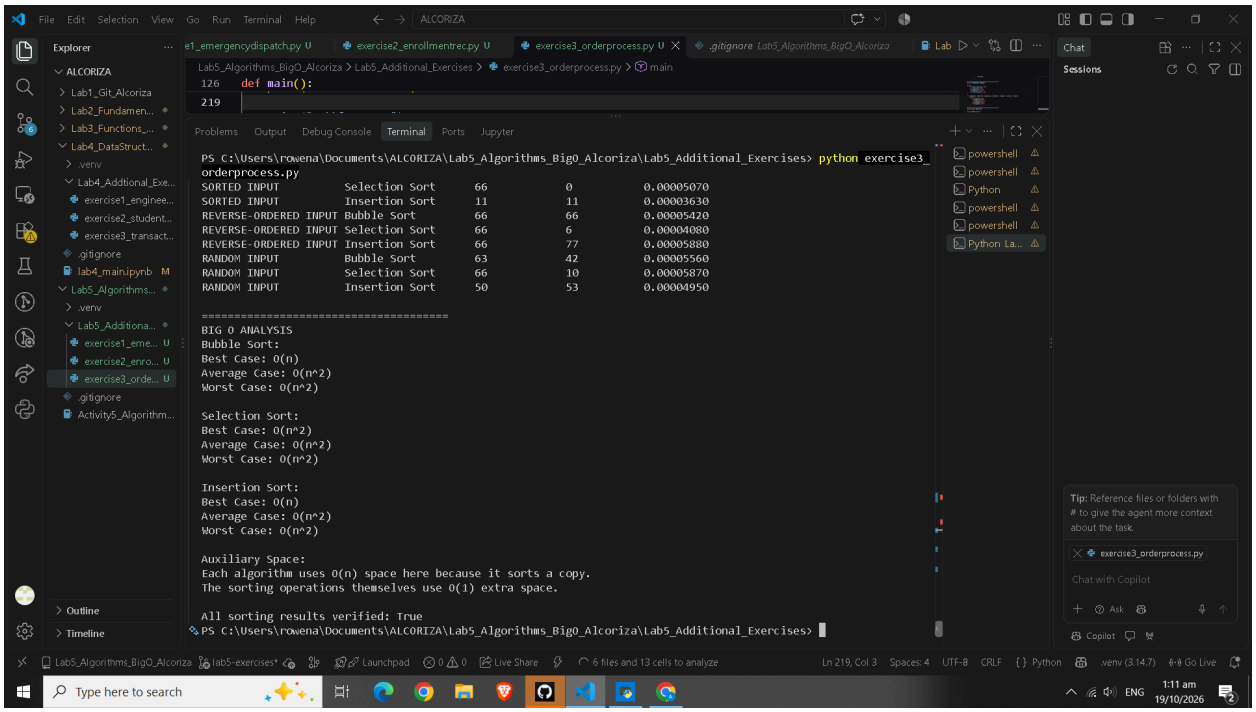
RESULTS


PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise2_enrollmentrec.py
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise3_orderprocess.py
Enter your last name: ALCORIZA
Enter your student ID: TUPM-26-1085
Enter your favorite artist: OLIVIA RODRIGO
STUDENT-SPECIFIC INPUTS
LAST_NAME: ALCORIZA
STUDENT_ID: TUPM-26-1085
FAVORITE_ARTIST: OLIVIA RODRIGO
SEED_DIGIT: 5
ID_SUM: 22
NAME_LENGTH: 8
ARTIST_LENGTH: 13
ARTIST_SUM: 154
E-COMMERCE ORDER PROCESSING
Generated Order Dataset: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
SORTED INPUT
Input: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Algorithm: Bubble Sort
Sorted Output: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Comparisons: 11
Movements: 0
Execution Time: 0.00002250 seconds
Sorting Verification: True
Algorithm: Selection Sort
Sorted Output: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Comparisons: 66


def main():
126
219
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise3_orderprocess.py
Algorithm: Selection Sort
Sorted Output: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Comparisons: 66
Movements: 0
Execution Time: 0.00005070 seconds
Sorting Verification: True
Algorithm: Insertion Sort
Sorted Output: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Comparisons: 11
Movements: 11
Execution Time: 0.00003630 seconds
Sorting Verification: True
==========
REVERSE-ORDERED INPUT
Input: [203, 201, 199, 197, 195, 193, 191, 189, 187, 185, 183, 181]
Algorithm: Bubble Sort
Sorted Output: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Comparisons: 66
Movements: 66
Execution Time: 0.00005420 seconds
Sorting Verification: True
Algorithm: Selection Sort
Sorted Output: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Comparisons: 66
Movements: 6
Execution Time: 0.00004080 seconds
Sorting Verification: True


def main():
126
219
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise3_orderprocess.py
Algorithm: Insertion Sort
Sorted Output: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Comparisons: 66
Movements: 77
Execution Time: 0.00005880 seconds
Sorting Verification: True
==========
RANDOM INPUT
Input: [193, 201, 195, 203, 187, 183, 191, 199, 181, 197, 185, 189]
Algorithm: Bubble Sort
Sorted Output: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Comparisons: 63
Movements: 42
Execution Time: 0.00005560 seconds
Sorting Verification: True
Algorithm: Selection Sort
Sorted Output: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Comparisons: 66
Movements: 10
Execution Time: 0.00005870 seconds
Sorting Verification: True
Algorithm: Insertion Sort
Sorted Output: [181, 183, 185, 187, 189, 191, 193, 195, 197, 199, 201, 203]
Comparisons: 50
Movements: 53
Execution Time: 0.00004950 seconds
Sorting Verification: True


def main():
126
219
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise3_orderprocess.py
PERFORMANCE SUMMARY
Input Condition Algorithm Comparisons Movements Time (seconds)
SORTED INPUT Bubble Sort 11 0 0.00002250
SORTED INPUT Selection Sort 66 0 0.00005070
SORTED INPUT Insertion Sort 11 11 0.00003630
REVERSE-ORDERED INPUT Bubble Sort 66 66 0.00005420
REVERSE-ORDERED INPUT Selection Sort 66 6 0.00004080
REVERSE-ORDERED INPUT Insertion Sort 66 77 0.00005880
RANDOM INPUT Bubble Sort 63 42 0.00005560
RANDOM INPUT Selection Sort 66 10 0.00005870
RANDOM INPUT Insertion Sort 50 53 0.00004950
BIG O ANALYSIS
Bubble Sort:
Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)
Selection Sort:
Best Case: O(n^2)
Average Case: O(n^2)
Worst Case: O(n^2)
Insertion Sort:
Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)
Auxiliary Space:
Each algorithm uses O(n) space here because it sorts a copy.
The sorting operations themselves use O(1) extra space.


def main():
126
219
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises> python exercise3_orderprocess.py
SORTED INPUT Selection Sort 66 0 0.00005070
SORTED INPUT Insertion Sort 11 11 0.00003630
REVERSE-ORDERED INPUT Bubble Sort 66 66 0.00005420
REVERSE-ORDERED INPUT Selection Sort 66 6 0.00004080
REVERSE-ORDERED INPUT Insertion Sort 66 77 0.00005880
RANDOM INPUT Bubble Sort 63 42 0.00005560
RANDOM INPUT Selection Sort 66 10 0.00005870
RANDOM INPUT Insertion Sort 50 53 0.00004950
BIG O ANALYSIS
Bubble Sort:
Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)
Selection Sort:
Best Case: O(n^2)
Average Case: O(n^2)
Worst Case: O(n^2)
Insertion Sort:
Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)
Auxiliary Space:
Each algorithm uses O(n) space here because it sorts a copy.
The sorting operations themselves use O(1) extra space.
All sorting results verified: True
PS C:\Users\rowena\Documents\ALCORIZA\Lab5_Algorithms_BigO_Alcoriza\Lab5_Additional_Exercises>